Промты для Шага 0

GEO-аудит · Цепочка разработки на Python · 01.11.2026

Восемь последовательных промптов из чата: от архитектуры и контрактов до проверки безопасности, оптимизации и передачи данных в шаг 1. Тексты промптов сохранены без содержательных изменений.

Как пользоваться

Отправляй промпты по одному в coding-чат с доступом к C:\GEO. Каждый этап должен завершаться работающим результатом и проверками. Следующий промпт продолжает работу с фактическим состоянием проекта.

Основной алгоритм: C:\GEO\output\pdf\Шаг 0 - Проверка адреса сайта.pdf. Редактируемый источник: C:\GEO\docs\steps\STEP_0_ALGORITHM_v1.md. Вход следующего этапа описан в C:\GEO\docs\steps\STEP_1_ALGORITHM_v1.md.

Последовательность

1. Архитектура и контракты данных.
2. Нормализация URL и правила принятия решений.
3. Защищённый сетевой слой.
4. Полный шаг 0, хранение и CLI.
5. Проверка корректности и устойчивости.
6. Отдельная проверка безопасности кода.
7. Оптимизация по измерениям.
8. Передача в шаг 1 и финальная приёмка.

Что должно получиться

Импортируемый Python-пакет и CLI для шага 0, воспроизводимые проверки, отчёт о безопасности, измеренная оптимизация, site.json и доказательства, а также типизированный вход шага 1 без повторной загрузки входного HTML.

Документ содержит задания для будущего выполнения. Его создание не означает, что эти задания уже выполнены. Фактическое состояние реализации определяется кодом и проектным контекстом.

ПРОМПТ 1 ИЗ 8 / ТЕКСТ ДЛЯ КОПИРОВАНИЯ

1. Архитектура и контракты данных

Разработай основу реализации шага 0 GEO-аудита на Python.

Рабочая папка: C:\GEO.

Прочитай:

- C:\GEO\AGENTS.md;
- C:\GEO\docs\PROJECT_CONTEXT.md;
- C:\GEO\docs\steps\STEP_0_ALGORITHM_v1.md;
- C:\GEO\output\pdf\Шаг 0 - Проверка адреса сайта.pdf;
- раздел входных данных в C:\GEO\docs\steps\STEP_1_ALGORITHM_v1.md.

Реализуем только шаг 0. Документы описывают требования к будущей программе. Их примеры JSON и псевдокод требуют полноценной реализации и проверки. Исходные PDF не изменяй.

Сначала изучи существующие файлы проекта. Сохрани полезный код и инструменты подготовки документов.

Подготовь архитектуру небольшого Python-пакета geo_audit с CLI и импортируемым API. Используй Python 3.12+, HTTPX и типизированные контракты. Выбери совместимые зависимости и зафиксируй их воспроизводимым способом.

Раздели ответственность:

1. Конфигурация и модели данных.
2. Нормализация URL и правила переходов.
3. Проверка сетевого назначения и HTTP-транспорт.
4. Управление бюджетом и временем.
5. Оркестрация шага 0.
6. Хранение доказательств.
7. Передача результата шагу 1.
8. CLI.

Выбирай минимальную архитектуру, которая позволяет независимо проверять эти обязанности. Обоснуй асинхронную или синхронную модель. Не добавляй инфраструктуру, которой задача пока не требует.

Определи модели:

- Step0Config;
- SiteValidationResult;
- FetchAttempt и RedirectHop;
- EvidenceRecord;
- BudgetSnapshot;
- Step1Input.

Зафиксируй:

- все состояния и условия can_enter_step1;
- обязательные поля и допустимость null;
- различие транспортной ошибки, HTTP-ошибки и непригодного содержимого;
- версии схем и конфигурации;
- инварианты и формат ошибок;
- сериализуемый выход для шага 1.

Создай архитектурную записку, таблицу соответствия требованиям алгоритма, каркас пакета, модели и схемы JSON. Не создавай заглушки, изображающие успешные сетевые проверки.

Проверь импорт, валидацию примеров и CLI --help. В конце сообщи, что создано, какие решения приняты, что проверено и что остаётся реализовать.

ПРОМПТ 2 ИЗ 8 / ТЕКСТ ДЛЯ КОПИРОВАНИЯ

2. Нормализация URL и правила принятия решений

Продолжи реализацию шага 0 в C:\GEO. Прочитай проектный контекст, актуальный алгоритм и результаты предыдущего этапа.

Реализуй чистые функции нормализации URL, проверки области и выбора следующего действия. Реальную сеть пока не подключай.

Требования:

- сохранить исходный ввод;
- поддержать HTTP/HTTPS, адрес без схемы и IDN;
- сохранить регистр пути и значимый query;
- удалить fragment;
- отклонить credentials, управляющие символы, неподдерживаемые схемы, некорректные хосты и порты;
- исключить неоднозначное декодирование адреса разными компонентами;
- отделить host, origin, entry_path и разрешённую область.

Реализуй таблицы решений из алгоритма:

- HTTPS-first при отсутствии схемы;
- первичная проверка явно указанной схемы;
- отдельный HTTP fallback только для разрешённых причин;
- отсутствие fallback после HTTP-статусов и барьеров доступа;
- отсутствие бессмысленной смены схемы при подтверждённом NXDOMAIN;
- остановка перекрёстных циклов HTTP/HTTPS;
- точные статусы ok, partial, needs_input, failed.

Автоматический www-алиас разрешай только по наблюдаемому редиректу и корректной проверке доменной границы. Используй локальный PSL с учётом private-секции. Другие поддомены не разрешай автоматически.

Оформи правила как проверяемую логику с явными входами и результатами. Не размазывай одинаковые решения по CLI и транспорту.

Добавь параметризованные тесты на реальные пограничные случаи, включая IDN, percent-encoding, query, относительный Location, нестандартные представления IP, www и многопользовательские домены.

Проверь инварианты нормализации и отсутствие изменений пользовательского намерения. Исправь найденные дефекты. Покажи результаты тестов и любые оставшиеся неоднозначности.

ПРОМПТ 3 ИЗ 8 / ТЕКСТ ДЛЯ КОПИРОВАНИЯ

3. Защищённый сетевой слой

Продолжи разработку в C:\GEO по актуальному алгоритму шага 0.

Реализуй общий HTTP-клиент, пригодный для повторного использования шагом 1.

Обязательные свойства:

- потоковый GET;
- TLS verification;
- ручная обработка редиректов;
- проверка назначения до каждого соединения;
- явная политика прокси, cookies и переменных окружения;
- ограниченные повторы;
- Retry-After;
- сетевые таймауты и отдельный общий deadline;
- лимиты полученных и распакованных данных;
- закрытие соединений при ошибке, отмене и превышении лимита.

Соблюдай начальные пределы алгоритма: 60 секунд и 15 HTTP-попыток для шага 0 внутри общего бюджета шагов 0-1. Redirect и retry расходуют попытки. Исключи скрытые повторы HTTP-транспорта.

Защита назначения должна учитывать:

- IPv4 и IPv6;
- private, loopback, link-local, reserved и metadata-адреса;
- IPv4-mapped IPv6;
- DNS-ответы и смену адреса;
- каждый редирект;
- повторное использование соединения;
- выбранный proxy-маршрут.

Один DNS precheck не считать защитой от DNS rebinding. Свяжи проверенное назначение с фактическим соединением через корректный transport/egress-механизм. Сохрани проверку hostname сертификата и SNI при подключении к проверенному адресу.

Выбирай поддерживаемые точки расширения библиотек. Не создавай собственную реализацию TLS. Если требуемый режим безопасности технически не обеспечен, возвращай явную ошибку вместо небезопасного продолжения.

Добавь офлайн-тесты транспорта и отдельные контролируемые интеграционные проверки границ соединения. MockTransport сам по себе не доказывает защиту реального подключения.

Проверь отсутствие запроса к запрещённому redirect-назначению, корректность счётчиков, ограничения распаковки и завершение медленного потока по deadline.

В конце опиши реализованный механизм защиты, доказательства его работы и ограничения.

ПРОМПТ 4 ИЗ 8 / ТЕКСТ ДЛЯ КОПИРОВАНИЯ

4. Полный шаг 0, хранение и CLI

Продолжи работу в C:\GEO. Собери полноценный шаг 0 из реализованных компонентов.

Реализуй validate_site() с инъекцией клиента, конфигурации, бюджета и средств времени. Функция возвращает SiteValidationResult; библиотека не использует input().

Добавь классификацию ответа:

- HTTP-статус и тип содержимого;
- полноценный HTML;
- JS-кандидат;
- пустое или бинарное содержимое;
- предполагаемая заглушка;
- подтверждённый барьер доступа.

Эвристические подозрения должны содержать основания и уровень определённости. Код 200 не считать достаточным признаком пригодности. Малый объём текста не считать доказательством ошибки.

Сохраняй:

- site.json;
- попытки и redirect-цепочки;
- исходные тела ответов;
- raw-манифест с SHA-256;
- конфигурацию и состояние бюджета;
- понятные причины и next_action.

Различай content-encoded bytes, распакованный HTML и декодированный текст. Не называй автоматически распакованное тело исходными сжатыми байтами.

Записывай доказательства до ссылающихся на них результатов. Используй атомарную публикацию файлов и отдельный каталог запуска. Обработай сбой записи и прерывание процесса. Не допускай положительного результата с отсутствующим обязательным доказательством.

Имена файлов формируй из внутренних ID. В логах редактируй секреты и чувствительные query; исходный полный URL храни только в предназначенных для этого локальных данных.

Добавь CLI запуска шага 0 с URL, каталогом вывода, настройками fallback и разрешённых хостов. Определи exit codes. needs_input должен возвращать конкретное действие без зависания в консоли.

Проверь сквозные сценарии успеха, HTTP fallback, внешнего редиректа, непригодного содержимого и ошибки хранения. Шаг 1 и платные API не реализуй.

ПРОМПТ 5 ИЗ 8 / ТЕКСТ ДЛЯ КОПИРОВАНИЯ

5. Проверка корректности и устойчивости

Проведи системную проверку реализации шага 0 в C:\GEO.

Сверь код с актуальным алгоритмом и таблицей требований. Для каждого требования укажи реализующий компонент и проверяющий сценарий. Не ограничивайся процентом покрытия.

Проверь:

1. HTTPS и явный HTTP.
2. Все разрешённые и запрещённые причины fallback.
3. TLS failure с успешным HTTP.
4. HTTP, ведущий обратно на неисправный HTTPS.
5. NXDOMAIN и временный DNS-сбой.
6. Относительные редиректы, циклы, лимиты и внешние хосты.
7. 200 challenge, JS-оболочку, 204, бинарный ответ.
8. Отсутствующий/неверный Content-Type и повреждённую кодировку.
9. Таймауты, Retry-After и исчерпание бюджета.
10. Слишком большое тело, распаковку и медленный поток.
11. Сбой диска, прерывание записи и повреждённые доказательства.
12. Согласованность статусов, can_enter_step1 и next_action.

Используй контролируемые фикстуры и транспорт. Для времени инъецируй clock/sleeper, чтобы проверки не зависели от длительных реальных ожиданий.

Для критичных отрицательных сценариев проверяй отсутствие нежелательного действия: запрос не отправлен, небезопасный файл не открыт, бюджет не увеличен. Проверки текста ошибки недостаточно.

Программные дефекты не должны маскироваться как недоступность сайта. Отмена выполнения должна освобождать ресурсы и сохранять завершённые наблюдения.

Исправь найденные ошибки и добавь регрессионные проверки. Сформируй отчёт о соответствии требованиям, выполненных тестах и оставшихся ограничениях. Не объявляй непроведённые интеграционные проверки успешными.

ПРОМПТ 6 ИЗ 8 / ТЕКСТ ДЛЯ КОПИРОВАНИЯ

6. Отдельная проверка безопасности кода

Проведи security review фактической реализации шага 0 в C:\GEO. Проверяй код и поведение независимо от заверений предыдущих отчётов.

Сначала составь краткую модель угроз: недоверенный URL, DNS, redirect, HTTP-заголовки, тело ответа, сжатые данные, пути артефактов и повторная загрузка результатов.

Проверь:

- SSRF и DNS rebinding на уровне соединения;
- обход ограничений через IPv6, альтернативные IP-представления и URL parsing differences;
- редиректы и случайное расширение allowed_hosts;
- proxy/environment bypass;
- TLS/SNI/hostname verification;
- decompression bombs и неограниченное потребление памяти;
- медленные ответы и обход deadline;
- утечки credentials, cookies, query и лог-инъекции;
- path traversal, symlink/reparse-point escape и подмену файлов;
- небезопасную десериализацию;
- гонки при записи и несогласованные артефакты;
- зависимости и ненужные привилегии.

Запусти подходящие статические проверки и аудит зависимостей. Проверь актуальные способы запуска выбранных инструментов. Вывод сканера проверь вручную: он не заменяет анализ сетевой архитектуры.

Для каждой подтверждённой проблемы укажи:

- серьёзность;
- файл и место;
- условия эксплуатации;
- воспроизводящий безопасный тест;
- исправление;
- результат регрессионной проверки.

Исправь подтверждённые дефекты в пределах шага 0. Неподтверждённые подозрения и ограничения окружения обозначь отдельно.

Не отключай проверки безопасности ради прохождения тестов. Не делай вывод «код полностью безопасен» на основании отсутствия предупреждений сканера.

Сохрани security-review с конкретными результатами, версиями инструментов и известными ограничениями.

ПРОМПТ 7 ИЗ 8 / ТЕКСТ ДЛЯ КОПИРОВАНИЯ

7. Оптимизация по измерениям

Оптимизируй реализацию шага 0 в C:\GEO после проверки корректности и безопасности.

Сначала измерь текущую реализацию на воспроизводимых сценариях:

- обычный HTTPS HTML;
- несколько редиректов;
- TLS failure и HTTP fallback;
- ответ около предельного размера;
- временные ошибки и ограниченные повторы.

Зафиксируй время, пиковую память, число DNS/HTTP-операций и объём лишних преобразований. Сетевые задержки в тестах должны быть контролируемыми.

Найди конкретные источники издержек:

- повторные запросы;
- повторное DNS-разрешение без необходимости;
- повторное чтение PSL и конфигурации;
- лишние копии тела;
- избыточное декодирование;
- повторное вычисление хэшей;
- неоправданные зависимости и абстракции.

Оптимизируй только подтверждённые проблемы. Учитывай потоковое сохранение и хэширование, разумное переиспользование клиента и передачу готового ответа шагу 1.

Сохрани семантику алгоритма, TLS verification, проверку назначения, границы scope, доказательства, rate limits, Retry-After и общие бюджеты. Кэш DNS не должен обходить политику безопасности. Не повышай скорость увеличением нагрузки на сайт.

После изменений повтори те же измерения и релевантные проверки корректности и безопасности. Покажи сравнение до/после и объясни компромиссы. Если значимого улучшения нет, сохрани более простой вариант.

Оптимизация не должна незаметно менять контракт Step1Input или смысл сохранённых статусов.

ПРОМПТ 8 ИЗ 8 / ТЕКСТ ДЛЯ КОПИРОВАНИЯ

8. Передача в шаг 1 и финальная приёмка

Заверши шаг 0 в C:\GEO и подготовь надёжный вход для будущего шага 1.

Прочитай входные требования в актуальном алгоритме шага 1. Реализуй типизированный Step1Input и загрузчик сохранённого результата.

Поддержи два режима:

1. Передача внутри одного процесса с готовым FetchResult и общим Budget.
2. Загрузка из каталога запуска через site.json и raw-манифест.
Контракт должен передавать:

- версии схемы, алгоритма и конфигурации;
- run_id;
- input_url, normalized_url, effective_url и origin;
- разрешённые хосты и область пути;
- status, can_enter_step1 и причины;
- выбранный исходный HTML, кодировку и полноту получения;
- evidence_id, хэши и историю попыток;
- TLS-наблюдения;
- использованный и оставшийся бюджет.
Загрузчик проверяет версии, согласованность полей, наличие файлов, хэши и допустимость путей внутри каталога запуска. При повреждении возвращает INPUT_ARTIFACT_INVALID. Не подставляй пустой HTML вместо отсутствующего доказательства.

Учти, что монотонное время нельзя напрямую переносить между процессами. Опиши и реализуй сериализуемую политику дедлайна: загрузка артефакта не обнуляет общий бюджет. Истёкший запуск требует явно определённого нового запуска, а не скрытого продления.

can_enter_step1 не является разрешением robots. Загрузчик сам не начинает обход.

Добавь интеграционную проверку: шаг 0 сохраняет результат, затем потребитель шага 1 загружает его и получает тот же HTML и ограничения без сетевого запроса. Проверь ok, допустимый partial, needs_input, failed, повреждённый файл, неизвестную версию и истёкший бюджет. Сам обход шага 1 пока не реализуй.

Выполни финальные тесты, проверку типов, линтер и применимые security checks. Подготовь README с командами установки и запуска в Windows PowerShell, описанием выходов и ограничений.

В итоговом отчёте дай:

- архитектуру и публичный API;
- команды запуска;
- пример выхода и входа шага 1;
- результаты тестов и security review;
- измерения оптимизации;
- оставшиеся ограничения.
Обнови PROJECT_CONTEXT.md по фактически выполненной работе. Исходные PDF сохрани.